

Competitive Programming

Section 2 Editorial: Simulation and brute force

Competitive Programming Course

2026-07-04

Table of contents

1	How to read this editorial	4
2	Problem A: Robot Walk	5
2.1	Restatement	5
2.2	Important constraints and traps	5
2.3	Observations	5
2.4	Sample walk-through	6
2.5	Algorithm	6
2.6	Correctness	6
2.7	Complexity	7
2.8	C++ solution	7
2.9	Python solution	7
2.10	Common wrong answers	8
2.11	Tests that matter	8
3	Problem B: Ticket Split	9
3.1	Restatement	9
3.2	Important constraints and traps	9
3.3	Observations	9
3.4	Candidate count	10
3.5	Sample walk-through	10
3.6	Algorithm	10
3.7	Correctness	11
3.8	Complexity	11
3.9	C++ solution	11
3.10	Python solution	12
3.11	Common wrong answers	12
3.12	Tests that matter	13
4	Problem C: Best Subset	14
4.1	Restatement	14
4.2	Important constraints and traps	14
4.3	Why greedy fails	14
4.4	Bit-mask representation	14
4.5	Sample walk-through	15
4.6	Algorithm	15
4.7	Correctness	16
4.8	Complexity	16
4.9	C++ solution	16
4.10	Python solution	17
4.11	Common wrong answers	17
4.12	Tests that matter	18
5	Stress testing strategy	19

1 How to read this editorial

This section is about disciplined simplicity. The accepted algorithms are not advanced, but they still need:

- a correct interpretation of the state;
- loop bounds justified by constraints;
- deterministic output when the judge expects one answer;
- boundary tests for zeros, equality, and empty choices.

When upsolving, do not skip the proof just because the code is short. Easy simulation and brute force problems are where you build the habits that later support binary search, greedy proofs, dynamic programming, and graph search.

2 Problem A: Robot Walk

2.1 Restatement

A robot starts at (0, 0). Each character in the command string moves it by one unit in one of four directions. For each command string, output:

```
final x-coordinate  
final y-coordinate  
maximum Manhattan distance ever reached
```

The maximum includes the starting position. Since the starting distance is 0, this matters mostly for the initialization.

2.2 Important constraints and traps

The total length of all command strings is at most 200000, so a single scan over every command is the intended solution.

The traps are not performance traps. They are state traps:

- mixing up x and y;
- making U decrease y;
- computing the maximum only at the end;
- forgetting that the path may return to the origin after going far away.

For URDL, the final position is the origin, but the path reached distance 2.

2.3 Observations

The current state is completely described by two integers:

```
x = horizontal coordinate  
y = vertical coordinate
```

Each command changes exactly one coordinate by exactly one.

The Manhattan distance after a move is:

$$|x| + |y|.$$

Therefore the solution is a direct simulation.

2.4 Sample walk-through

For `s = URDL`:

step	command	x	y	distance	best
0	start	0	0	0	0
1	U	0	1	1	1
2	R	1	1	2	2
3	D	1	0	1	2
4	L	0	0	0	2

The output is:

```
0 0 2
```

2.5 Algorithm

```
for each test case:
    x = 0
    y = 0
    best = 0
    for command in s:
        apply command to x or y
        best = max(best, abs(x) + abs(y))
    print x, y, best
```

2.6 Correctness

We prove that the algorithm prints the required values for one command string.

After processing the first `i` commands, maintain this invariant:

`(x, y)` is the robot's position after exactly the first `i` commands, and `best` is the maximum Manhattan distance among the starting position and all positions reached after those `i` commands.

Before any command is processed, the robot is at `(0, 0)` and the maximum distance seen is 0, so the invariant is true.

Assume it is true after `i` commands. The next command moves the robot exactly as specified by the statement: one unit up, down, left, or right. Therefore the new `(x, y)` is the correct position after `i + 1` commands. The algorithm then computes the distance of this new position and updates `best` with the maximum of the old `best` and the new distance. Thus `best` is the maximum distance over all positions seen through `i + 1` commands. The invariant is preserved.

After all commands are processed, the invariant says that `(x, y)` is the final position and `best` is the maximum distance ever reached. The algorithm prints exactly these values.

2.7 Complexity

Let L be the length of the command string. The algorithm does $O(L)$ work and uses $O(1)$ memory. Across the input, the total work is linear in the total command length.

2.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        string s;
        cin >> s;
        int x = 0;
        int y = 0;
        int best = 0;
        for (char ch : s) {
            if (ch == 'U') {
                ++y;
            } else if (ch == 'D') {
                --y;
            } else if (ch == 'L') {
                --x;
            } else {
                ++x;
            }
            best = max(best, abs(x) + abs(y));
        }
        cout << x << ' ' << y << ' ' << best << '\n';
    }
    return 0;
}
```

2.9 Python solution

```
import sys

def solve_case(s: str) -> tuple[int, int, int]:
    x = 0
```

```

y = 0
best = 0
for ch in s:
    if ch == "U":
        y += 1
    elif ch == "D":
        y -= 1
    elif ch == "L":
        x -= 1
    else:
        x += 1
    best = max(best, abs(x) + abs(y))
return x, y, best

data = sys.stdin.buffer.read().split()
t = int(data[0])
out = []
for i in range(1, t + 1):
    x, y, best = solve_case(data[i].decode())
    out.append(f"{x} {y} {best}")
sys.stdout.write("\n".join(out))

```

2.10 Common wrong answers

- Updating `best` only after the whole string.
- Treating U as `x += 1`.
- Forgetting `abs` in the Manhattan distance.
- Initializing `best` to a negative value and mishandling very short strings.

2.11 Tests that matter

single command:	U
return to origin:	LR, UD, URDL
one direction only:	LLLL
go far then return:	RRRLLLL
mixed movement:	RDRD

3 Problem B: Ticket Split

3.1 Restatement

For each test case, find nonnegative integers x , y , and z such that:

$$\begin{aligned}x + y + z &= n \\ a*x + b*y + c*z &= s\end{aligned}$$

The output must be deterministic: check x from 0 to n , and for each x , check y from 0 to $n - x$. Print the first valid triple in that scan order. If no triple exists, print -1 .

3.2 Important constraints and traps

The key bound is $n \leq 300$. This makes $O(n^2)$ enumeration safe.

The traps are:

- using three loops when two are enough;
- forgetting that a count may be zero;
- allowing z to become negative;
- printing a different valid triple when the judge expects the first one;
- checking only the cost equation and forgetting the ticket count equation.

3.3 Observations

The equation

$$x + y + z = n$$

means that once x and y are chosen, z is forced:

$$z = n - x - y$$

So we do not need to loop over z . We only need to ensure that $z \geq 0$, which is handled by looping $y \leq n - x$.

The cost check becomes:

$$a*x + b*y + c*(n - x - y) == s$$

3.4 Candidate count

For a fixed n , the number of pairs (x, y) with $x \geq 0$, $y \geq 0$, and $x + y \leq n$ is:

$$1 + 2 + \dots + (n + 1) = \frac{(n + 1)(n + 2)}{2}.$$

For $n = 300$, this is:

$$\frac{301 \cdot 302}{2} = 45451.$$

That is small.

3.5 Sample walk-through

Consider:

```
n = 5, s = 17, a = 2, b = 3, c = 5
```

The scan begins:

x	y	z	cost	valid?
0	0	5	25	no
0	1	4	23	no
0	2	3	21	no
0	3	2	19	no
0	4	1	17	yes

The required output is:

```
0 4 1
```

There may be another valid triple later, but the statement asks for the first one in this scan order.

3.6 Algorithm

```
for each test case:
    found = false
    for x from 0 to n:
        for y from 0 to n - x:
            z = n - x - y
            if a*x + b*y + c*z == s:
                print x, y, z
                found = true
```

```

        stop searching this case
    if not found:
        print -1

```

3.7 Correctness

First, we prove coverage. Any feasible triple has nonnegative x , y , and z , and satisfies $x + y + z = n$. Therefore $0 \leq x \leq n$. For that fixed x , we have $0 \leq y \leq n - x$. The loops include exactly those ranges. When the loops reach this x and y , the algorithm computes:

```
z = n - x - y
```

which is the same z from the feasible triple. Thus every feasible triple is considered.

Second, we prove selection. The algorithm checks candidates in the required order: increasing x , and for each x , increasing y . It prints the first candidate whose cost equals s . By coverage, if any feasible triple exists, it will be considered. Because the algorithm stops at the first feasible candidate in the required order, it prints the required triple. If the loops finish without printing, no feasible triple was considered, and by coverage no feasible triple exists. Printing -1 is then correct.

3.8 Complexity

The nested loops check $O(n^2)$ pairs. Each check is $O(1)$. Memory is $O(1)$.

With $n \leq 300$, this is comfortably within the limit.

3.9 C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, s, a, b, c;
        cin >> n >> s >> a >> b >> c;
        bool found = false;
        for (int x = 0; x <= n && !found; ++x) {
            for (int y = 0; y + x <= n; ++y) {
                int z = n - x - y;
                if (a * x + b * y + c * z == s) {

```

```

        cout << x << ' ' << y << ' ' << z << '\n';
        found = true;
        break;
    }
}
}
if (!found) {
    cout << "-1\n";
}
}
return 0;
}

```

3.10 Python solution

```

import sys

def find_split(n: int, s: int, a: int, b: int, c: int):
    for x in range(n + 1):
        for y in range(n - x + 1):
            z = n - x - y
            if a * x + b * y + c * z == s:
                return x, y, z
    return None

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n, s, a, b, c = data[idx:idx + 5]
    idx += 5
    ans = find_split(n, s, a, b, c)
    if ans is None:
        out.append("-1")
    else:
        out.append(f"{ans[0]} {ans[1]} {ans[2]}")
sys.stdout.write("\n".join(out))

```

3.11 Common wrong answers

- Using `range(n)` instead of `range(n + 1)`, which excludes `n`.
- Letting `y` run to `n` and producing negative `z`.
- Printing the first solution found by a different loop order.
- Ignoring cases with `n = 0`.

- Assuming costs are positive; this exercise allows zero costs.

3.12 Tests that matter

```
n = 0, s = 0:           0 0 0
one ticket, each type works: catches order and z derivation
all costs equal:       many valid triples, first one matters
impossible high sum:   -1
zero-cost tickets:     verifies nonnegative counts
```

4 Problem C: Best Subset

4.1 Restatement

Given up to 20 item values and a limit s , choose any subset whose total is at most s . Print the largest possible total.

The empty subset is allowed, so 0 is always feasible.

4.2 Important constraints and traps

The constraint $n \leq 20$ is the signal. All subsets can be enumerated:

$$2^{20} = 1048576.$$

That is large enough to require care, but small enough for this direct exercise.

The traps are:

- excluding the empty subset;
- using a greedy strategy such as taking largest items first;
- stopping when a subset exceeds the limit, even though a different subset may be feasible;
- using 32-bit integers for sums involving 10^9 values;
- misunderstanding that item order does not matter.

4.3 Why greedy fails

Suppose:

```
s = 10
values = 8 5 5
```

Taking the largest item first gives total 8. But the best subset is $5 + 5 = 10$.

Because local choices can block better combinations, exhaustive search is the right first solution at this constraint size.

4.4 Bit-mask representation

For `values = [3, 4, 8, 5]`, the mask 1010 means:

```
bit 0: 0 -> do not take 3
bit 1: 1 -> take 4
bit 2: 0 -> do not take 8
bit 3: 1 -> take 5
```

The total is $4 + 5 = 9$.

Every subset corresponds to exactly one mask from 0 to $2^n - 1$.

4.5 Sample walk-through

For:

```
n = 4, s = 10
values = 3 4 8 5
```

some masks are:

mask	chosen values	total	feasible?
0000	empty	0	yes
0001	3	3	yes
0011	3, 4	7	yes
0100	8	8	yes
1010	4, 5	9	yes
1100	8, 5	13	no
1111	3, 4, 8, 5	20	no

The best feasible total is 9.

4.6 Algorithm

```
best = 0
for mask from 0 to  $2^n - 1$ :
    total = 0
    for i from 0 to  $n - 1$ :
        if bit i is set in mask:
            total += values[i]
    if total <= s:
        best = max(best, total)
print best
```

4.7 Correctness

Every subset of the n items has a unique bit mask: bit i is set exactly when item i is included. The loop over masks from 0 to $2^n - 1$ therefore considers every possible subset exactly once.

For each mask, the inner loop adds exactly the values whose bits are set, so `total` is the sum of the represented subset. If `total` $\leq s$, the subset is feasible. The algorithm updates `best` to the maximum feasible total seen so far.

Initially, `best` = 0, which is the value of the empty subset and is feasible. After all masks have been processed, every feasible subset has been considered, and `best` is at least its total. Since `best` is only updated using feasible totals, it is itself feasible. Therefore `best` is exactly the largest achievable total not exceeding s .

4.8 Complexity

There are 2^n masks. For each mask, the inner loop checks n bits. The time complexity is:

$$O(n2^n).$$

The memory usage is $O(1)$ besides the input array.

For $n = 20$, this is about 20 million bit checks.

4.9 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        long long limit;
        cin >> n >> limit;
        vector<long long> values(n);
        for (long long &value : values) {
            cin >> value;
        }

        long long best = 0;
        for (int mask = 0; mask < (1 << n); ++mask) {
            long long total = 0;
            for (int i = 0; i < n; ++i) {
```

```

        if (mask & (1 << i)) {
            total += values[i];
        }
    }
    if (total <= limit) {
        best = max(best, total);
    }
}
cout << best << '\n';
}
return 0;
}

```

4.10 Python solution

```

import sys

def best_subset(values: list[int], limit: int) -> int:
    n = len(values)
    best = 0
    for mask in range(1 << n):
        total = 0
        for i, value in enumerate(values):
            if mask & (1 << i):
                total += value
        if total <= limit and total > best:
            best = total
    return best

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    limit = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    out.append(str(best_subset(values, limit)))
sys.stdout.write("\n".join(out))

```

4.11 Common wrong answers

- Starting the mask loop at 1, which excludes the empty subset.

- Assuming the best subset has consecutive items.
- Sorting and greedily taking items while possible.
- Using `int` for totals in C++ when values may be large.
- Using `mask <= (1 << n)` and accidentally including an invalid extra mask.

4.12 Tests that matter

```
n = 0:                answer 0
limit = 0:            only zero-valued subset or empty subset
all items too large:  answer 0
single item equals limit: exact boundary
greedy counterexample: 8 5 5 with limit 10
large values:         checks 64-bit sums
```

5 Stress testing strategy

Simulation and brute force problems are excellent for randomized testing.

For Problem A, random command strings catch direction mistakes and forgotten maximum updates. A useful generator should include:

- one-direction strings such as LLLLL;
- strings that return to the origin;
- mixed strings;
- short strings of length 1.

For Problem B, generated cases should include both guaranteed-solvable and possibly-impossible inputs. One way to guarantee a solution is:

```
choose random x, y, z with  $x + y + z = n$   
set  $s = a*x + b*y + c*z$ 
```

For Problem C, random tests should include small n , $n = 0$, limit 0, limit equal to the sum of all items, and values that create greedy failures.

The key lesson is that random testing is most useful when the generator is biased toward edge cases, not only average-looking inputs.

6 What to carry forward

This section gives you three reusable habits:

1. In simulation, define the state before writing updates.
2. In brute force, compute the candidate count before trusting the loops.
3. In testing, use brute force as a reference whenever the optimized solution is harder than the small version.

The next sections will keep using these ideas. Prefix sums, sorting, two pointers, BFS, and dynamic programming all become easier when you can first write the direct process and reason about its state.